

Week5_예습과제_진예원

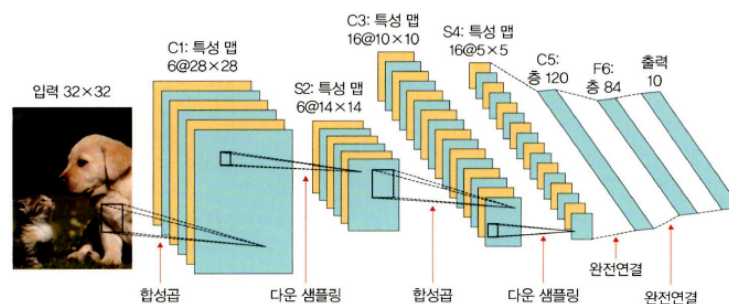
6 합성곱 신경망 II

6.1 이미지 분류를 위한 신경망

- 입력 데이터로 이미지를 사용한 분류 = 특정 대상이 영상 내에 존재하는지 여부를 판단하는 것
- 이미지 분류(image classification)에서 주로 사용되는 합성곱 신경망 유형 알아보자

6.1.1 LeNet-5

- 합성곱 신경망 개념을 최초로 얀 르쿤이 개발한 구조
 - 1995년 얀 르쿤, 레옹 보토, 요슈아 벤지오, 패트릭 하프너가 수표에 쓴 손글씨 숫자를 인식하는 딥러닝 구조 LeNet-5를 발표 - 현재 CNN의 초석이 됨
- 합성곱(convolutional) & 다운 샘플링(혹은 풀링)(sub-sampling) 반복적으로 거치면서 마지막에 완전연결층에서 분류를 수행함

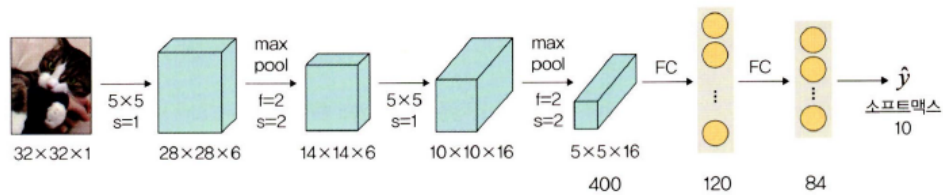


- C1 - 5×5 합성곱 연산 $\rightarrow 28 \times 28$ 크기 특성 맵 6개 생성
- S2 - 다운 샘플링 $\rightarrow$ 특성 맵 크기 14×14 로 축소
- C3 - 5×5 합성곱 연산 $\rightarrow 10 \times 10$ 크기 특성 맵 16개 생성
- S4 - 다운 샘플링 $\rightarrow$ 특성 맵 크기 5×5 로 축소
- C5 - 5×5 합성곱 연산 $\rightarrow 1 \times 1$ 크기 특성 맵 120개 생성
- F6 - 완전연결층 $\rightarrow$ C5 결과를 유닛(노드) 84개에 연결
 - C : 합성곱층

- S : 풀링층
- F : 완전연결층

LeNet-5 예제

- 32×32 크기 이미지에 합성곱층 & 최대 풀링층 쌍으로 두 번 적용 → 완전연결층 거쳐 이미지가 분류되는 신경망



▼ 표 6-1 LeNet-5 예제 신경망 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	32×32	—	—	—
합성곱층	6	28×28	5×5	1	렐루(ReLU)
최대 풀링층	6	14×14	2×2	2	—
합성곱층	16	10×10	5×5	1	렐루(ReLU)
최대 풀링층	16	5×5	2×2	2	—
완전연결층	—	120	—	—	렐루(ReLU)
완전연결층	—	84	—	—	렐루(ReLU)
완전연결층	—	2	—	—	소프트맥스(softmax)

기본 데이터셋 준비

0. 아나콘다 프롬프트에서 tqdm 라이브러리 설치

```
pip instal --user tqdm
conda install -c conda-forge tqdm
#둘 중 하나로 설치
```

- tqdm: progress(진행상태) 의미. 진행 상태를 바(bar) 형태로 가시화하여 보여줌. 주로 모델 훈련에 대한 진행상태 확인할 때 사용

1. 라이브러리 호출

```
import torch
import torchvision
```

```

from torch.utils.data import DataLoader, Dataset
from torchvision import transforms #이미지 변환(전처리) 기능 제공
from torch.autograd import Variable
from torch import optim #경사하강법 이용하여 가중치 구함
import torch.nn as nn
import torch.nn.functional as F
import os #파일 경로에 대한 함수들 제공
import cv2
from PIL import Image
from tqdm import tqdm_notebook as tqdm #진행상황 가시적으로 표현해줌. 특히 모델 학습 경과 확인할 때 사용
import random
from matplotlib import pyplot as plt

device = torch.device("cuda: 0" if torch.cuda.is_available() else "cpu")
#파이토치 텐서플로와 다르게 GPU 자동으로 할당 X -> GPU 할당을 모델과 데이터에 선언해줘야함.
#단 이 장에서는 CPU 사용

```

2. 모델 학습에 필요한 데이터셋 전처리(텐서 변환 등)

```

class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]),
            'val': transforms.Compose([
                transforms.Resize(256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),

```

```

        transforms.Normalize(mean, std)
    ])
}

def __call__(self, img, phase):
    return self.data_transform[phase](img)

```

- 토치비전 라이브러리 이용하면 이미지에 대한 전처리 쉽게 가능.
torchvision.transforms에서 사용하는 파라미터
 - transforms.Compose: 이미지를 변형할 수 있는 방식들의 묶음
 - transforms.RandomResizedCrop: 입력 이미지를 주어진 크기(resize: 224 × 244)로 조정
 - scale: 원래 이미지를 임의의 크기만큼(0.5~1.0(50~100%) 면적을 무작위로 자르겠다
 - transforms.RandomHorizontalFlip: 주어진 확률로 이미지를 수평 반전시킴
 - 이때 확률값 지정 X - 기본값 0.5 확률(훈련 이미지 중 반은 위아래 뒤집힌 상태, 반은 그대로 사용)
 - transforms.ToTensor: ImageFolder: 메서드를 비롯해서 torchvision 메서드는 이미지를 읽을 때 파이썬 이미지 라이브러리인 PIL 사용함
 - PIL 사용해서 이미지 읽으면 생성되는 이미지 범위 [0, 255] / 배열의 차원이 높이 H x 너비 W x 채널 수 C로 표현됨
 - 이후 효율적인 연산을 위해 torch.FloatTensor 배열로 바뀌어야 하는데, 이때 픽셀 값의 범위 [0.0, 1.0] / 차원의 순서로 채널 수 C x 높이 H x 너비 W로 바뀜
 - 이 작업을 ToTensor() 메서드가 해줌
 - transforms.Normalize: 전이학습에서 사용하는 사전 훈련된 모델들은 보통 ImageNet 데이터셋에서 훈련되었음 → 사전 훈련된 모델 사용하기 위해서는 ImageNet 데이터의 각 채널별 평균과 표준편차에 맞는 정규화를 해줘야함.
 - Normalize 메서드 안에 사용된 (mean: 0.485, 0.456, 0.406), (std: 0.229, 0.224, 0.225): ImageNet에서 이미지의 RGB 채널마다 평균과 표준편차 의미
 - cf) OpenCV 사용해서 이미지 읽어 오면 RGB 이미지가 아닌 BGR 이미지 → 채널 순서 주의

- `__call__`: 클래스를 호출할 수 있도록 하는 메서드. `__init__` 인스턴스 초기화를 위해 사용. `__call__` 인스턴스 호출되었을 때 실행됨 → 클래스에 `__call__` 있을 경우 클래스 객체 자체 호출하면 `__call__` 함수의 리턴 값이 반환됨

3. 이미지 데이터셋을 불러온 후 훈련, 검증, 테스트로 분리

```
cat_directory = r'/content/chap06/data/dogs-vs-cats/Cat/'
dog_directory = r'/content/chap06/data/dogs-vs-cats/Dog/'

cat_images_filepaths = sorted([os.path.join(cat_directory,
f) for f in
                                os.listdir(cat_directory)])
dog_images_filepaths = sorted([os.path.join(dog_directory,
f) for f in
                                os.listdir(dog_directory)])
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths]
correct_images_filepaths = [i for i in images_filepaths if
cv2.imread(i) is not None]

random.seed(42)
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400] #훈련용 400개의 이미지
val_images_filepaths = correct_images_filepaths[400:-10] #검증용 92개의 이미지
test_images_filepaths = correct_images_filepaths[-10:] #테스트용 열 개의 이미지
print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))
```

- 고양이 이미지 데이터를 가져옴
 - `sorted`: 데이터를 정렬된 리스트로 만들어서 반환
 - `os.path.join`: 경로와 파일명을 결합하거나 분할된 경로를 하나로 합치고 싶을 때 사용
 - `cat_directory` 디렉터리와 `os.listdir`을 통해 검색된 이미지 파일들(`f`)을 하나로 합쳐서 `'../chap06/data/dogs-vs-cats/Cat/이미지 파일명'`으로 표시

- 경로 합는 다른 예시

```
import os
list_path = ['C:\\', 'Temp', 'user']
folder_path = os.path.join(*list_path)
folder_path
```

> 'C:\\Temp\\user' (경로가 하나로 합쳐짐)

- os.listdir: 지정한 디렉터리 내 모든 파일의 리스트 반환. 예제의 Cat 디렉터리의 이미지 파일들을 모두 반환함
- images_filepaths에서 이미지 파일들을 불러옴
 - for 반복문 이용해서 가져온 데이터에 대해 i를 이용해서 리스트로 생성
 - for 반복문 이용하여 images_filepaths에서 이미지 데이터 검색
 - if 조건문 의미. cv2.imread() 함수를 이용하여 모든 이미지 데이터를 읽어옴(not None 상태, 더 이상 데이터를 찾을 수 없을 때까지를 의미)
- 넘파이 random() 함수는 임의의 난수 생성. 이때 난수를 생성하기 위해 사용되는 것은 시드 값(seed value) / Numpy.random.seed() 메서드는 상태를 초기화함
 - 이 모듈이 호출될 때마다 임의의 난수 재생성됨 but 특정 시드 값 부여하면 상태가 저장되기 때문에 동일한 난수 생성함
 - 예시

```
#시드 값 101로 설정
import numpy as np
np.random.seed(101)
np.random.randint(low=1, high=10, size=10)
```

> array([2, 7, 8, 9, 5, 9, 6, 1, 6, 9])

```
#시드 값 100으로 설정
import numpy as np
np.random.seed(101)
np.random.randint(low=1, high=10, size=10)
```

> array([9, 9, 4, 8, 8, 1, 5, 3, 6, 3])

```
#다시 시드 값 101로 설정
np.random.seed(101)
np.random.randint(low=1, high=10, size=10)
```

```
> array([2, 7, 8, 9, 5, 9, 6, 1, 6, 9])
```

4. 테스트 데이터셋 이미지 확인 함수

- 주어진 데이터셋 훈련, 검증 테스트 용도로 분리했으므로 이젠 테스트 용도의 데이터셋에 어떤 데이터들이 있는지 확인

```
def display_image_grid(images_filepaths, predicted_labels=
(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize
=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        true_label = os.path.normpath(image_filepath).split(os.
sep)[-2]
        predicted_label = predicted_labels[i] if predicted_labe
ls else true_label
        color = "green" if true_label == predicted_label else
"red"
        #예측 = 정답(레이블)이면 초록색 표시, 아니면 빨간색 표시
        ax.ravel()[i].imshow(image) #개별 이미지 출력
        ax.ravel()[i].set_title(predicted_label, color=color) #
predicted_label을 타이틀로 사용
        ax.ravel()[i].set_axis_off() #이미지의 축 제거
    plt.tight_layout() #이미지 여백 조정
    plt.show()
```

- cv2.cvtColor: 이미지 색상 변경
 - 첫 번째 파라미터: 입력 이미지
 - 두 번째 파라미터: 변환할 이미지 색상 지정. BGR 채널 이미지를 RGB로 변경
- 이미지의 전체 경로를 정규화하고 분할을 위한 코드

- `os.path.normpath`: 경로명 정규화
 - `A//B, A/B/, A/./, A/./B` → `A/B`로 경로 통일
- `split(os.sep)`: 경로를 / 혹은 \를 기준으로 분할할 때 사용
 - `c:/temp/user/a.jpg` 경로에 `split(os.sep)` → `['c:', 'temp', 'user', 'a.jpg']`로 분할됨
 - `split(os.sep)[-2]` → `user` 반환
- `predicted_label`에 대한 값 정의하기 위한 코드
 - `predicted_labels` 값이 있으면 그 값을 `predicted_label`로 사용
 - `predicted_labels` 값이 없다면 `true_label` 값을 `predicted_label`로 사용

5. 테스트 데이터셋 이미지 출력

```
display_image_grid(test_images_filepaths)
```

모델 학습 단계

- 데이터셋에는 학습할 데이터 경로 정의, 그 경로에서 데이터 읽어옴
 - 데이터셋 크기 클 수 O → `__init__`에서 전체 데이터 읽어오지 x 경로만 지정해놓고, `__getitem__` 메서드에서 이미지 읽어옴
→ 데이터를 어디에서 가져올지 결정
- 데이터로더에서 데이터셋의 데이터를 메모리로 불러옴
 - 한꺼번에 전체 데이터 불러오는 것 x 배치크기만큼 분할하여 가져옴
- `DogvsCatDataset()` 클래스 - 데이터 불러오는 방법 정의
- 이번 예제 목저기 다수의 개와 고양이 이미지가 포함된 데이터에서 이들을 예측
→ 레이블(정답) 이미지에서 고양이와 개가 포함될 확률을 코드로 구현(ex. 고양이 있는 이미지의 레이블은 0, 개가 있는 이미지 레이블은 1)

6. 이미지 데이터셋 클래스 정의

```
class DogvsCatDataset(Dataset):
    def __init__(self, file_list, transform=None, phase='train'):
```

```

self.file_list = file_list
self.transform = transform
self.phase = phase #train 적용

def __len__(self): #images_filepaths 데이터셋 전체 길이 반환
    return len(self.file_list)

def __getitem__(self, idx):
    img_path = self.file_list[idx]
    img = Image.open(img_path)
    img_transformed = self.transform(img, self.phase)
    label = img_path.split('/')[-1].split('.')[0]
    if label == 'dog':
        label = 1
    elif label == 'cat':
        label = 0
    return img_transformed, label

```

- 이미지 데이터에 대한 레이블 값 (dog, cat) 가져옴
 - img_path: 이미지가 위치한 전체 경로 보여줌
 - 이미지 데이터의 레이블(dog)을 가져오려면 '/'와 '.'을 제거해야함
 - img_path.split('/')[-1]: 이미지의 전체 경로에서 '/' 제거
 - split('.')[0]: 마지막으로 img_path.split('/')[-1]을 통해 얻은 결과에서 '.'을 제거

7. 변수 값 정의

```

size = 224
mean = (0.485, 0.456, 0.406)
std = (0.22, 0.224, 0.225)
batch_size = 32

```

8. 이미지 데이터셋 정의

```

train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_dataloader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
dataloader_dict = {'train': train_dataloader, 'val': val_dataloader}

batch_iterator = iter(train_dataloader)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)

```

9. 데이터로더 정의

```

train_dataset = DogvsCatDataset(train_images_filepaths, transform=ImageTransform(size, mean, std), phase='train')
val_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size, mean, std), phase='val')

train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_dataloader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
dataloader_dict = {'train': train_dataloader, 'val': val_dataloader}

batch_iterator = iter(train_dataloader)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)

```

- 파이토치의 데이터로더 - 배치 관리 담당. 한 번에 모든 데이터를 불러오면 메모리에 부담. 데이터를 그룹으로 쪼개서 조금씩 불러온다
 - 첫 번째 파라미터: 데이터를 불러오기 위한 데이터셋
 - batch_size: 한 번에 메모리로 불러올 데이터 크기(여기선 32)

- shuffle: 메모리로 데이터를 가져올 때 임의로 섞음

10. 모델의 네트워크 클래스

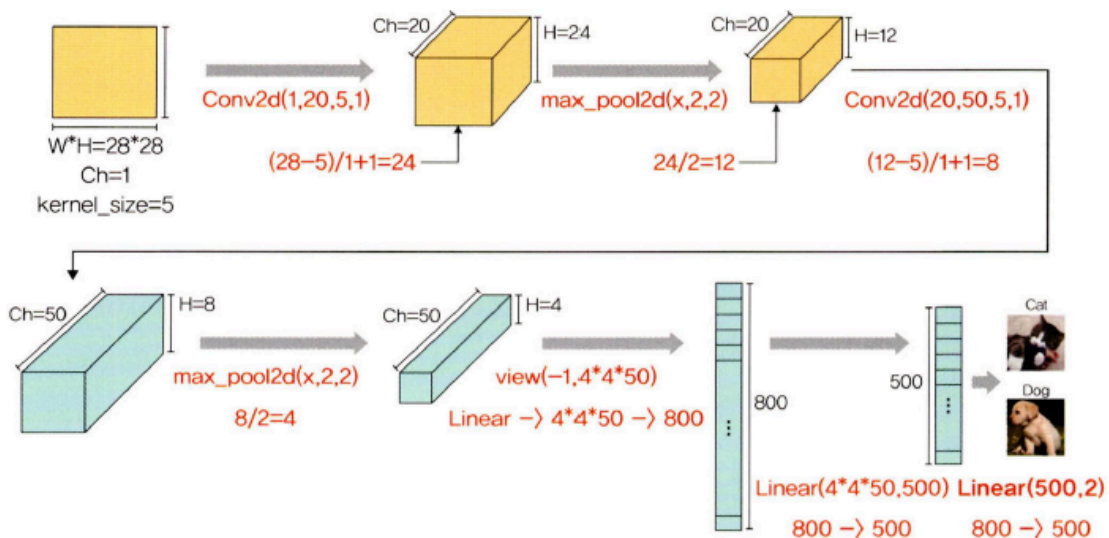
```
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.cnn1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=5, stride=1, padding=0)
        self.relu1 = nn.ReLU()
        self.maxpool1 = nn.MaxPool2d(kernel_size=2)
        self.cnn2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=5, stride=1, padding=0)
        self.relu2 = nn.ReLU() # activation
        self.maxpool2 = nn.MaxPool2d(kernel_size=2)
        self.fc1 = nn.Linear(32*5*5, 512)
        self.relu5 = nn.ReLU()
        self.fc2 = nn.Linear(512, 2)
        self.output = nn.Softmax(dim=1)

    def forward(self, x):
        out = self.cnn1(x)
        out = self.relu1(out)
        out = self.maxpool1(out)
        out = self.cnn2(out)
        out = self.relu2(out)
        out = self.maxpool2(out)
        out = out.view(out.size(0), -1)
        out = self.fc1(out)
        out = self.relu5(out)
        out = self.fc2(out)
        out = self.output(out)
        return out
```

- Conv2d 계층에서 출력 크기 구하는 공식 - 출력 크기 = $(W-F+2P)/S+1$
 - W: 입력 데이터의 크기(input_volume_size)
 - F: 커널 크기(kernel_size)
 - P: 패딩 크기(padding_size)

- S: 스트라이드(strides)
- MaxPool2d 계층에서의 출력 크기 구하는 공식 - 출력 크기 = IF/F
 - IF: 입력 필터의 크기(input_filter_size=앞의 Conv2d 출력 크기)
 - F: 커널 크기(kernel_size)

▼ 그림 6-5 합성곱층과 풀링층의 크기(형태) 계산



11. 모델 객체 생성

```
model = LeNet().to(device)
print(model)
```

```
LeNet(
  (cnn1): Conv2d(3, 16, kernel_size=(5, 5), stride=(1, 1))
  (relu1): ReLU()
  (maxpool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (cnn2): Conv2d(16, 32, kernel_size=(5, 5), stride=(1, 1))
  (relu2): ReLU()
  (maxpool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=89888, out_features=512, bias=True)
  (relu5): ReLU()
  (fc2): Linear(in_features=512, out_features=2, bias=True)
  (output): Softmax(dim=1)
)
```

12. torchsummary 라이브러리를 이용한 모델의 네트워크 구조 확인

- 11에서의 출력 결과가 한 눈에 들어오지 않는다면 torchsummary 라이브러리 사용해 확인 가능

- torchsummary: 케라스와 같은 형태로 모델 출력

```
from torchsummary import summary
summary(model, input_size=(3, 224, 224))
```

- summary 통해 모델의 네트워크 관련 정보 확인 가능
 - 첫 번째 파라미터: 모델의 네트워크
 - 두 번째 파라미터: 입력 값으로 (채널, 너비, 높이)가 된다

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 16, 220, 220]	1,216
ReLU-2	[-1, 16, 220, 220]	0
MaxPool2d-3	[-1, 16, 110, 110]	0
Conv2d-4	[-1, 32, 106, 106]	12,832
ReLU-5	[-1, 32, 106, 106]	0
MaxPool2d-6	[-1, 32, 53, 53]	0
Linear-7	[-1, 512]	46,023,168
Linear-8	[-1, 2]	1,026
Softmax-9	[-1, 2]	0
Total params: 46,038,242		
Trainable params: 46,038,242		
Non-trainable params: 0		
Input size (MB): 0.57		
Forward/backward pass size (MB): 19.47		
Params size (MB): 175.62		
Estimated Total Size (MB): 195.67		

- 출력 결과
 - 11에 비해 네트워크 내 파라미터 수와 구조가 이해하기 쉽게 표현됨
 - 출력되는 정보: 총 파라미터 수, 입력 크기, 네트워크 총 크기 등

13. 학습 가능한 파라미터 수 확인

```
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
```

```
print(f'The model has {count_parameters(model):,} trainable parameters')
```

> The model has 46,038,242 trainable parameters

14. 옵티마이저와 손실 함수 정의

```
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
criterion = nn.CrossEntropyLoss()
```

- 경사 하강법으로 모멘텀 SGD 사용
 - 모멘텀 SGD: 관성이 추가된 것. 매번 기울기를 구하지만 가중치 수정 전에 이전 수정 방향(+,-)을 참고하여 같은 방향으로 일정한 비율만 수정되게 하는 방법
 - 첫 번째 파라미터: 경사 하강법을 통해 궁극적으로 업데이트하고자 하는 파라미터: 가중치, 바이어스
 - lr(learning rate): 가중치를 변경할 때 얼마나 크게 변경할지 결정
 - momentum: SGD를 적절한 방향으로 가속화하여 흔들림(진동) 줄여줌

15. 모델의 파라미터와 손실 함수를 CPU에 할당

- 파이토치 - GPU 할당 자동화 x → 모델의 모든 파라미터 및 손실 함수가 GPU 를 사용할 수 있도록 지정해줘야함. But 아직 GPU 활성화 X때문에 여기서는 CPU 사용

```
model = model.to(device)
criterion = criterion.to(device)
```

16. 모델 학습 함수 정의

- 학습 용도 → model.train() 사용

```
def train_model(model, dataloader_dict, criterion, optimizer, num_epoch):
    since = time.time()
    best_acc = 0.0
```

```

for epoch in range(num_epoch):
    print('Epoch {}/{}'.format(epoch + 1, num_epoch))
    print('- '*20)

    for phase in ['train', 'val']:
        if phase == 'train':
            model.train()
        else:
            model.eval()

        epoch_loss = 0.0
        epoch_corrects = 0

        for inputs, labels in tqdm(dataloader_dict[phase]):

            inputs = inputs.to(device)
            labels = labels.to(device)
            optimizer.zero_grad()

            with torch.set_grad_enabled(phase == 'train'):

                outputs = model(inputs)
                _, preds = torch.max(outputs, 1)
                loss = criterion(outputs, labels)

                if phase == 'train':
                    loss.backward()
                    optimizer.step()

            epoch_loss += loss.item() * inputs.size(0) #1
            epoch_corrects += torch.sum(preds == labels.data)

        epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset)
        epoch_acc = epoch_corrects.double() / len(dataloader_dict[phase].dataset)

```

```

        print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))

    if phase == 'val' and epoch_acc > best_acc:
        best_acc = epoch_acc
        best_model_wts = model.state_dict()

    time_elapsed = time.time() - since
    print('Training complete in {:.0f}m {:.0f}s'.format(
        time_elapsed // 60, time_elapsed % 60))
    print('Best val Acc: {:.4f}'.format(best_acc))
    return model

```

- 오차와 입력 곱하는 이유 전에 손실 함수의 reduction 파라미터 이해 먼저
 - 코드 6-14에서 손실함수 정의할 때
 - reduction 파라미터 명시 X - 기본값 그대로 사용 = mean
 - mean: 정답과 예측 값의 오차를 구한 후 그 값들의 평균 반환

⇒ 손실 함수 특성상 전체 오차를 배치 크기로 나눔으로써 평균 반환하기 때문에, epoch_loss를 계산하는 동안 loss.item()과 inputs.size(0) 곱해준다

17. 모델 학습

- train_model 호출

```

import time

num_epoch = 10
model = train_model(model, dataloader_dict, criterion, optimizer, num_epoch)

```

```

Epoch 1/10
-----
/tmp/ipykernel_980/3689613115.py:18: TqdmDeprecationWarning: This function w
Please use `tqdm.notebook.tqdm` instead of `tqdm.tqdm_notebook`
    for inputs, labels in tqdm(dataloader_dict[phase]):
100% ██████████ 13/13 [00:51<00:00, 3.16s/it]
train Loss: 0.6985 Acc: 0.4750
100% ██████████ 3/3 [00:05<00:00, 1.55s/it]
val Loss: 0.7044 Acc: 0.4891
Epoch 2/10
-----
100% ██████████ 13/13 [00:33<00:00, 1.95s/it]
train Loss: 0.6879 Acc: 0.5175
100% ██████████ 3/3 [00:03<00:00, 1.22s/it]

Epoch 9/10
-----
100% ██████████ 13/13 [00:29<00:00, 2.06s/it]
train Loss: 0.6495 Acc: 0.6400
100% ██████████ 3/3 [00:03<00:00, 1.03s/it]
val Loss: 0.6760 Acc: 0.5652
Epoch 10/10
-----
100% ██████████ 13/13 [00:29<00:00, 2.03s/it]
train Loss: 0.6438 Acc: 0.6700
100% ██████████ 3/3 [00:03<00:00, 1.05s/it]
val Loss: 0.6767 Acc: 0.5652
Training complete in 5m 57s
Best val Acc: 0.565217

```

- 검증 데이터셋 이용한 모델 학습 결과 최고 56% 정확도 = 높지 않은 결과
 - 조금 더 높은 결과를 원한다면 데이터셋 늘려서 테스트

18. 모델 테스트 위한 함수 정의

- 테스트 데이터셋을 모델에 적용하여 정확도 측정
- 측정 결과는 데이터 프레임에 담아둔 후 CSV 파일로 저장
- 테스트 용도 데이터셋 이용 → `model.eval()` 사용

```

import pandas as pd

id_list = []
pred_list = []
_id=0
with torch.no_grad():

```

```

for test_path in tqdm(test_images_filepaths):
    img = Image.open(test_path)
    _id = test_path.split('/')[1].split('.')[1]
    transform = ImageTransform(size, mean, std)
    img = transform(img, phase='val')
    img = img.unsqueeze(0) #1
    img = img.to(device)

    model.eval()
    outputs = model(img)
    preds = F.softmax(outputs, dim=1)[:, 1].tolist() #2
    id_list.append(_id)
    pred_list.append(preds[0])

res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
})

res.sort_values(by='id', inplace=True)
res.reset_index(drop=True, inplace=True)

res.to_csv('LesNet.csv', index=False)

```

- torch.unsqueeze - 텐서에 차원 추가할 때 사용 + (0) 차원이 추가될 때 위치를 의미
 - (ex) 형태가 (3)인 텐서가 있다고 가정해보자. 0 위치에 차원을 추가하면 형태가 (1,3)이 된다 → 행 한 개와 열 세 개의 구조 갖는 텐서가 생성됨
 - 그렇다면 2D에 텐서가 추가될 때는 어떨까?
 - 형태가 (2,2)인 2D 텐서가 있을 때 0 위치에 차원 추가 → 텐서 모양 (1,2,2)가 됨 → 하나의 채널, 행 두 개와 열 두 개를 의미
 - 1 위치에 차원을 추가하면 (2,1,2)의 형태가 되므로 채널 두 개, 행 한 개, 열 두 개가 됨
 - 2 위치에 차원을 추가하면 (2,2,1)의 형태가 되므로 채널 두 개와 행 두 개, 열 한 개가 됨
- 소프트맥스 - 지정된 차원을 따라 텐서의 요소(텐서의 개별 값)가 (0,1) 범위에 있고 합계가 1이 되도록 크기 다시 조정

- outputs, dim=1: outputs에 softmax를 적용하여 각 행의 합이 1이 되도록 함
- [:, 1]: output, dim=1의 값 중 모든 행(:)에서 두 번째 칼럼(1번째 인덱스) 가져옴
- tolist(): 배열을 리스트 형태로 변환

```
for test_path in tqdm(test_images_filepaths):
```

100%  10/10 [00:00<00:00, 20.18it/s]

- 예측 결과를 CSV 파일로 저장하는 함수였기 때문에 실행 결과는 큰 의미가 없으며 단순히 처리가 되었다고 이해하면 됨

19. 테스트 데이터셋의 예측 결과 호

- 테스트 데이터셋을 모델에 적용한 결과를 LeNet 파일로 저장해두었으므로 그 결과를 살펴보자

```
res.head(10)
```

	id	label
0	109	0.437212
1	145	0.428945
2	15	0.574168
3	162	0.452964
4	167	0.515737
5	200	0.410415
6	210	0.596889
7	211	0.504929
8	213	0.393111
9	224	0.568463

- DataFrame에서 열 개의 값을 보여줌
- 예측 결과 레이블 > 0.5 → 개 의미 / < 0.5 → 고양이 의미

20. 테스트 데이터셋 이미지를 출력하기 위한 함수 정의

```

class_ = classes = {0:'cat', 1:'dog'}
def display_image_grid(images_filepaths, predicted_labels=
(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        a = random.choice(res['id'].values)
        label = res.loc[res['id'] == a, 'label'].values[0]
        if label > 0.5:
            label = 1
        else:
            label = 0
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
        ax.ravel()[i].set_axis_off()
    plt.tight_layout()
    plt.show()

```

21. 테스트 데이터셋 예측 결과 이미지 출력

```
display_image_grid(test_images_filepaths)
```

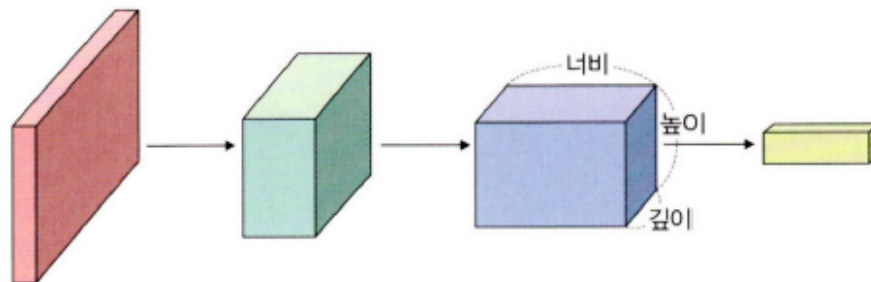


- 예측력이 좋진 X 특히 일부 데이터 이용한 모델 학습 진행했기 때문
- 캐글에서 내려받은 데이터 전체 사용한다면 성능 높아지겠지만 CPU 사용한다면 훈련 시간 오래 걸릴 수 있음

6.1.2 AlexNet

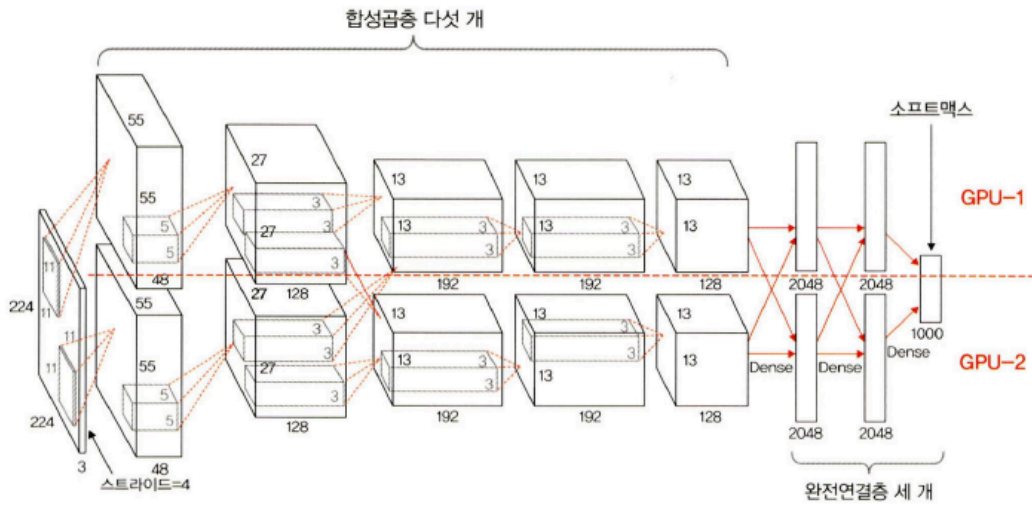
- ImageNet 영상 데이터베이스 기반 화상 인식 대회 ILSVRC 2012에서 우승한 CNN 구조
- CNN 구조 복습
 - 3차원 구조 - (이미지 크기) 너비, 높이, 깊이
 - 색상 이미지 = R/G/B로 시작이 3
 - but 합성곱 거치면서 특성 맵 생성되고, 이것에 따라 중간 영상의 깊이가 달라진다

▼ 그림 6-8 CNN 구조



- 기본 구조
 - GPU 2개 기반으로 한 병렬 구조
 - 합성곱층(Conv) 5개 + 완전연결층(FC) 3개 총 8개 계층으로 구성
 - 맨 마지막 완전연결층은 1000개 카테고리 분류 위한 소프트맥스 활성화 함수 사용
 - 학습 가능한 파라미터 수: 약 6,600만 개
 - 활성화 함수로 ReLU 사용

♥ 그림 6-9 AlexNet 구조



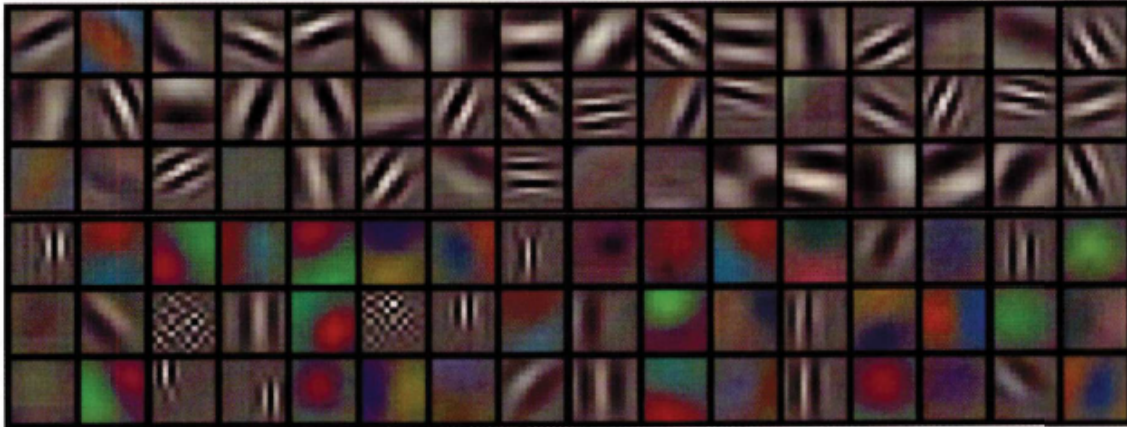
♥ 표 6-2 AlexNet 구조 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	227×227	—	—	—
합성곱층	96	55×55	11×11	4	렐루(ReLU)
최대 풀링층	96	27×27	3×3	2	—
합성곱층	256	27×27	5×5	1	렐루(ReLU)
최대 풀링층	256	13×13	3×3	2	—
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	256	13×13	3×3	1	렐루(ReLU)
최대 풀링층	256	6×6	3×3	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

- 입력 및 출력
 - 입력: 227 x 227 x 3 크기의 RGB 이미지
 - 출력: 각 클래스(카테고리)에 해당하는 1000 x 1 확률 벡터
- 첫 번째 합성곱층 및 GPU 동장
 - 커널 크기 11 x 11 x 3, 스트라이드 4 적용 → 특성 맵 96개 생성하므로 55 x 55 x 96 출력

- GPU-1에서는 주로 컬러와 상관없는 정보 추출 위한 커널 학습됨, GPU-2에서는 주로 컬러와 관련된 정보 추출 위한 커널이 학습됨

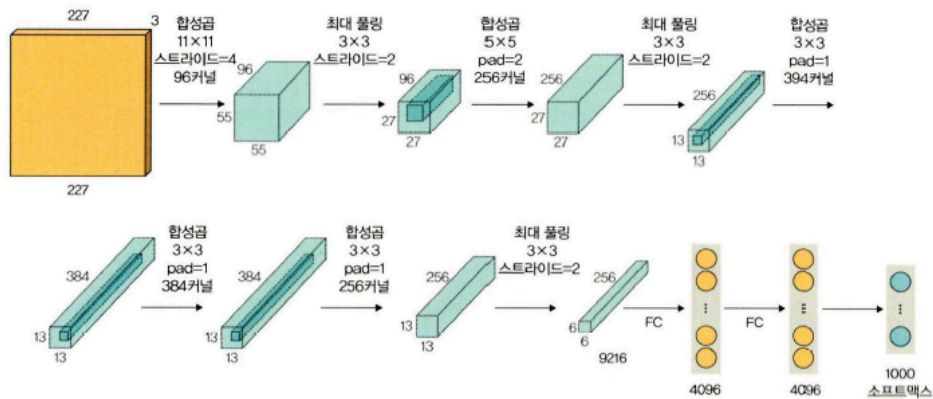
♥ 그림 6-10 AlexNet GPU-1 · 2 적용 결과



0. 필요한 라이브러리 호출

```
import torch
import torchvision
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms
from torch.autograd import Variable
from torch import optim
import torch.nn as nn
import torch.nn.functional as F
import os
import cv2
import random
import time
import matplotlib.pyplot as plt
from PIL import Image
from tqdm import tqdm_notebook as tqdm
device = torch.device("cuda:0" if torch.cuda.is_available()
else "cpu")
```

♥ 그림 6-11 AlexNet 예제 네트워크



2. 데이터 전처리

```
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=
(0.5, 1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]),
            'val': transforms.Compose([
                transforms.Resize(256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ])
        }

    def __call__(self, img, phase):
        return self.data_transform[phase](img)
```

3. 데이터를 가져와서 훈련, 검증, 테스트 용도로 분

```

cat_directory = r'/content/chap06/data/dogs-vs-cats/Cat/'
dog_directory = r'/content/chap06/data/dogs-vs-cats/Dog/'

cat_images_filepaths = sorted([os.path.join(cat_directory,
f) for f in os.listdir(cat_directory)])
dog_images_filepaths = sorted([os.path.join(dog_directory,
f) for f in os.listdir(dog_directory)])
images_filepaths = [*cat_images_filepaths, *dog_images_file
paths]
correct_images_filepaths = [i for i in images_filepaths if
cv2.imread(i) is not None]

random.seed(42)
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400]
val_images_filepaths = correct_images_filepaths[400:-10]
test_images_filepaths = correct_images_filepaths[-10:]
print(len(train_images_filepaths), len(val_images_filepath
s), len(test_images_filepaths))

```

4. 커스텀 데이터셋 정의

- torch.utils.data.Dataset을 상속받아 커스텀 데이터셋 정의

```

class DogvsCatDataset(Dataset):
    def __init__(self, file_list, transform=None, phase='tr
ain'):
        self.file_list = file_list
        self.transform = transform
        self.phase = phase

    def __len__(self):
        return len(self.file_list)

    def __getitem__(self, idx):
        img_path = self.file_list[idx]
        img = Image.open(img_path)

```

```

img_transformed = self.transform(img, self.phase)

label = img_path.split('/')[-1].split('.')[0]
if label == 'dog':
    label = 1
elif label == 'cat':
    label = 0

return img_transformed, label

```

5. 변수에 대한 값 정의

```

size = 256
mean = (0.486, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32

```

- 전처리에 필요한 평균(mean), 표준편차(std) 등에 대한 변수 값 정의

6. 훈련, 검증, 테스트 데이터셋 정의

- 앞에서 정의한 DogvsCatDataset(0 클래스에 훈련, 검증, 테스트 데이터를 적용하되 전처리도 함께하도록 한다

```

train_dataset = DogvsCatDataset(train_images_filepaths, transform=ImageTransform(size, mean, std), phase='train')
val_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size, mean, std), phase='val')
test_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size, mean, std), phase='val')

index = 0
print(train_dataset.__getitem__(index)[0].size())
print(train_dataset.__getitem__(index)[1])

```

- 출력 결과
torch.Size([3, 256, 256])

0

- 훈련 데이터셋 크기 (3, 256, 256) = (채널, 너비, 높이)

7. 데이터셋을 메모리로 불러옴

```
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_dataloader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_dataloader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
dataloader_dict = {'train': train_dataloader, 'val': val_dataloader}
```

```
batch_iterator = iter(train_dataloader)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)
```

```
torch.Size([32, 3, 256, 256])
tensor([[1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1,
        1, 1, 1, 0, 0, 0, 0, 0]])
```

8. AlexNet 모델 네트워크 정의

- 사전 훈련된 네트워크와 유사하게 정의
- 합성곱 + 활성화 함수 + 풀링이 다섯번 반복된 후 두 개의 완전연결층과 출력층으로 구성된 네트워크

```
class AlexNet(nn.Module):
    def __init__(self) -> None:
        super(AlexNet, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True), # 1
            nn.MaxPool2d(kernel_size=3, stride=2),
```

```

        nn.Conv2d(64, 192, kernel_size=5, padding=2),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=3, stride=2),
        nn.Conv2d(192, 384, kernel_size=3, padding=1),
        nn.ReLU(inplace=True),
        nn.Conv2d(384, 256, kernel_size=3, padding=1),
        nn.ReLU(inplace=True),
        nn.Conv2d(256, 256, kernel_size=3, padding=1),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=3, stride=2),
    )
    self.avgpool = nn.AdaptiveAvgPool2d((6, 6)) # 2
    self.classifier = nn.Sequential(
        nn.Dropout(),
        nn.Linear(256 * 6 * 6, 4096),
        nn.ReLU(inplace=True),
        nn.Dropout(),
        nn.Linear(4096, 512),
        nn.ReLU(inplace=True),
        nn.Linear(512, 2),
    )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x

```

- inplace 연산: 연산에 대한 결과값을 새로운 변수에 저장 X 기존 데이터를 대체하는 것
= 기존 값을 연산 결과값으로 대체하면서 기존 값들을 무시
- nn.AdaptiveAvgPool2d - nn.AvgPool2d처럼 풀링 위해 사용
 - AvgPool2d에서는 풀링에 대한 커널 및 스트라이드 크기를 정의해야 동작함
 - ex. nn.AvgPool2d((3,2), stride=(2,1)) 처럼 지정 → 그 결과는 5 x 5 텐서를 3 x 3 텐서로, 7 x 7 텐서를 4 x 4 텐서로 줄이는 효과를 얻음
 - AvgPool2d: (N, C, H_{in}, W_{in}) 크기 입력 → (N, C, H_{out}, W_{out}) 크기 출력

$$H_{out} = \left\lceil \frac{H_{in} + 2 \times padding[0] - kernel_size[0]}{stride[0]} + 1 \right\rceil$$

$$W_{out} = \left\lceil \frac{W_{in} + 2 \times padding[1] - kernel_size[1]}{stride[1]} + 1 \right\rceil$$

- AdaptiveAvgPool2d: 풀링 작업 끝날 때 필요한 출력 크기를 정의함
→ nn.AvgPool2d에서는 커널 크기, 스트라이드, 패딩을 지정했다면 nn.AdaptiveAvgPool2d는 출력에 대한 크기만 지정함
- H_{out} , W_{out} 값이 정해졌기 때문에 커널 크기, 스트라이드, 패딩 값을 구할 수 있음
- 따라서 AdaptiveAvgPool2d 경우 출력 크기에 대한 조정이 쉬워짐. 입력 크기에 변동이 있고 CNN 위쪽에 완전연결층을 사용하는 경우 유용함

9. model 객체 생성

```
model = AlexNet()
model.to(device)
```

10. 옵티마이저 및 손실 함수 정의

```
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
criterion = nn.CrossEntropyLoss()
```

11. 모델 네트워크 구조 확인

- torchsummary 이용하여 모델 네트워크 확인
- 모델 네트워크를 파라미터와 함께 보여주므로 내부적으로 어떤 일이 이루어지는지 예측하며 살펴볼 수 O

```
from torchsummary import summary
summary(model, input_size=(3, 256, 256))
```

Layer (type)	Output Shape	Param #
=====		
Conv2d-1	[-1, 64, 63, 63]	23,296
ReLU-2	[-1, 64, 63, 63]	0
MaxPool2d-3	[-1, 64, 31, 31]	0
Conv2d-4	[-1, 192, 31, 31]	307,392
ReLU-5	[-1, 192, 31, 31]	0
MaxPool2d-6	[-1, 192, 15, 15]	0
Conv2d-7	[-1, 384, 15, 15]	663,936
ReLU-8	[-1, 384, 15, 15]	0
Conv2d-9	[-1, 256, 15, 15]	884,992
ReLU-10	[-1, 256, 15, 15]	0
Conv2d-11	[-1, 256, 15, 15]	590,080
ReLU-12	[-1, 256, 15, 15]	0
MaxPool2d-13	[-1, 256, 7, 7]	0
AdaptiveAvgPool2d-14	[-1, 256, 6, 6]	0
Dropout-15	[-1, 9216]	0
Linear-16	[-1, 4096]	37,752,832
ReLU-17	[-1, 4096]	0
Dropout-18	[-1, 4096]	0
Linear-19	[-1, 512]	2,097,664
ReLU-20	[-1, 512]	0
Linear-21	[-1, 2]	1,026
=====		

Total params: 42,321,218
Trainable params: 42,321,218
Non-trainable params: 0

Input size (MB): 0.75
Forward/backward pass size (MB): 10.90
Params size (MB): 161.44
Estimated Total Size (MB): 173.10

12. 모델 학습 함수 정의

```
def train_model(model, dataloader_dict, criterion, optimizer, num_epoch):
    since = time.time()
    best_acc = 0.0

    for epoch in range(num_epoch):
        print('Epoch {}/{}'.format(epoch + 1, num_epoch))
        print('- '*20)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()
```

```

        else:
            model.eval()

        epoch_loss = 0.0
        epoch_corrects = 0

        for inputs, labels in tqdm(dataloader_dict[phase]):

            inputs = inputs.to(device)
            labels = labels.to(device)
            optimizer.zero_grad()

            with torch.set_grad_enabled(phase == 'train'):

                outputs = model(inputs)
                _, preds = torch.max(outputs, 1)
                loss = criterion(outputs, labels)

                if phase == 'train':
                    loss.backward()
                    optimizer.step()

            epoch_loss += loss.item() * inputs.size(0)

            epoch_corrects += torch.sum(preds == labels.data)

        epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset)
        epoch_acc = epoch_corrects.double() / len(dataloader_dict[phase].dataset)

        print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))

    time_elapsed = time.time() - since
    print('Training complete in {:.0f}m {:.0f}s'.format(

```

```

        time_elapsed // 60, time_elapsed % 60))
    return model

```

13. 모델 학습

- 훈련 위해 정의된 train_model() 함수 호출하여 모델 학습하자

```

num_epoch = 10
model = train_model(model, dataloader_dict, criterion, optimizer, num_epoch)

```

- 모델 학습 결과. 훈련과 검증 데이터셋이 적기 때문에 학습 결과가 좋지 않음

14. 모델을 이용한 예측

```

import pandas as pd

id_list = []
pred_list = []
_id=0
with torch.no_grad():
    for test_path in tqdm(test_images_filepaths):
        img = Image.open(test_path)
        _id = test_path.split('/')[1].split('.')[1]
        transform = ImageTransform(size, mean, std)
        img = transform(img, phase='val')
        img = img.unsqueeze(0)
        img = img.to(device)

        model.eval()
        outputs = model(img)
        preds = F.softmax(outputs, dim=1)[:, 1].tolist()

        id_list.append(_id)
        pred_list.append(preds[0])

res = pd.DataFrame({

```

```

        'id': id_list,
        'label': pred_list
    })
    res.to_csv('alexnet.csv', index=False)

```

- 모델 예측에 대한 실행 결과. 예측 결과를 CSV 파일로 저장하는 함수이므로 결과는 큰 의미 X, 단순히 처리가 완료된 결과를 보여줌

15. 데이터 프레임 결과 확인

```
res.head(10)
```

- label > 0.5 = 개, label < 0.5 = 고양이로 예측했음을 의미

16. 예측 결과를 시각적으로 표현하기 위한 함수 정의

```

class_ = classes = {0:'cat', 1:'dog'}
def display_image_grid(images_filepaths, predicted_labels=
(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsi
ze=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        a = random.choice(res['id'].values)
        label = res.loc[res['id'] == a, 'label'].values[0]
        if label > 0.5:
            label = 1
        else:
            label = 0
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
        ax.ravel()[i].set_axis_off()
    plt.tight_layout()
    plt.show()

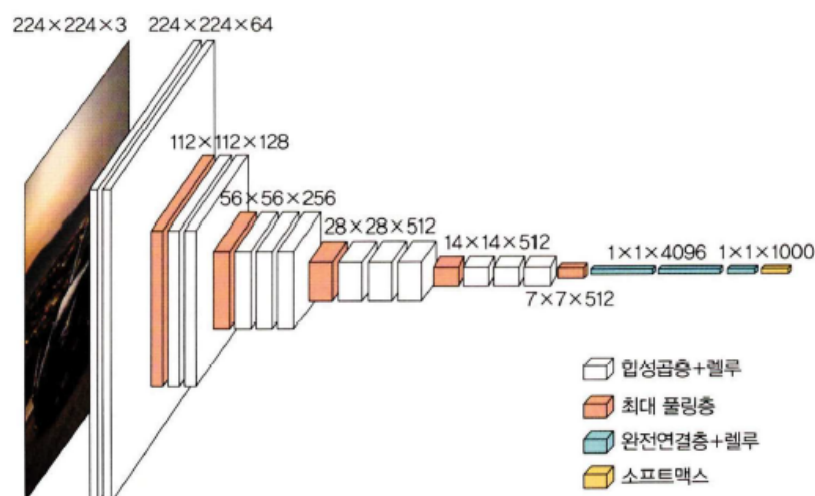
```

17. 예측 결과에 대해 이미지와 함께 출력

```
display_image_grid(test_images_filepaths)
```

6.1.3 VGGNet

- 카넬 시모니안 & 앤드류 지서만이 2015 ICLR에 게재한 "Very deep convolutional networks for large-scale image recognition" 논문에서 처음 발표함
- 탄생 배경: 합성곱층의 파라미터 수를 줄이고, 훈련 시간 개선하려고 탄생
→ 네트워크를 깊게 만드는 것이 성능에 어떤 영향을 미치는지 확인하고자 나온 모델
- 특징
 - 깊이의 영향만 최대한 확인하고자 합성곱층에서 사용하는 필터/커널의 크기를 가장 작은 3×3 으로 고정
 - 네트워크 계층의 총 개수에 따라 여러 유형의 VGGNet(VGG16, VGG19)이 있음
- VGG16 네트워크 구조
 - 파라미터 수: 총 1억 3300만 개
 - 모든 합성곱 커널의 크기 3×3 , 최대 풀링 커널 크기 2×2 , 스트라이드는 2
 - 결과적으로 64개의 224×224 특성맵($224 \times 224 \times 64$) 생성됨
 - 마지막 16번째 계층 제외하고 모두 ReLU 활성화 함수가 적용됨



♥ 표 6-3 VGG16 구조 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	224×224	—	—	—
합성곱층	64	224×224	3×3	1	렐루(ReLU)
합성곱층	64	224×224	3×3	1	렐루(ReLU)
최대 풀링층	64	112×112	2×2	2	—
합성곱층	128	112×112	3×3	1	렐루(ReLU)
합성곱층	128	112×112	3×3	1	렐루(ReLU)
최대 풀링층	128	56×56	2×2	2	—
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
최대 풀링층	256	28×28	2×2	2	—
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
최대 풀링층	512	14×14	2×2	2	—

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
최대 풀링층	512	7×7	2×2	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

- 이번 예제는 VGGNet 중 가장 간단한 VGG11을 파이토치로 구현함

0. 필요한 라이브러리 호출

```
import copy #1
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
```

```
import torchvision.transforms as transforms
import torchvision.datasets as Datasets
import matplotlib.pyplot as plt
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

- 객체 복사
 - 얕은 복사(shallow copy)
 - 깊은 복사(deep copy)
 - 예제

1. 기본 참조 복사 =

```
original = [1, 2, 3]
copy_o = original #copy_o에 original 복사(기본 참조 복사 =)
print(copy_o)
copy_o[2] = 10 #copy_o의 3을 10으로 값을 변경
print(copy_o)
print(original)
```

- 출력 결과

```
[1, 2, 3]
[1, 2, 10]
[1, 2, 10]
```

- copy_o뿐만 아니라 원래 값인 original의 3도 10으로 바뀜

2. 얕은 복사 copy.copy()

```
import copy

original = [[1, 2], 3] copy_o에 얕은 복사(copy.copy())
print(copy_o)
copy_o[0] = 100
print(copy_o)
print(original)

append = copy.copy(original)
```

```
append[0].append(4)
print(append)
print(original)
```

◦ 출력 결과

```
[[1, 2], 3]
[100, 3]
[[1, 2], 3]
[[1, 2, 4], 3]
[[1, 2, 4], 3]
```

- copy_o만 변경
- [1, 2]에 4를 추가했더니 original과 copy_o 모두 반영됨

3. 깊은 복사 copy.deepcopy()

```
import copy

original = [[1, 2], 3]
copy_o = copy.deepcopy(original)
print(copy_o)
copy_o[0] = 100
print(copy_o)
print(original)

append = copy.deepcopy(original)
append[0].append(4)
print(append)
print(original)
```

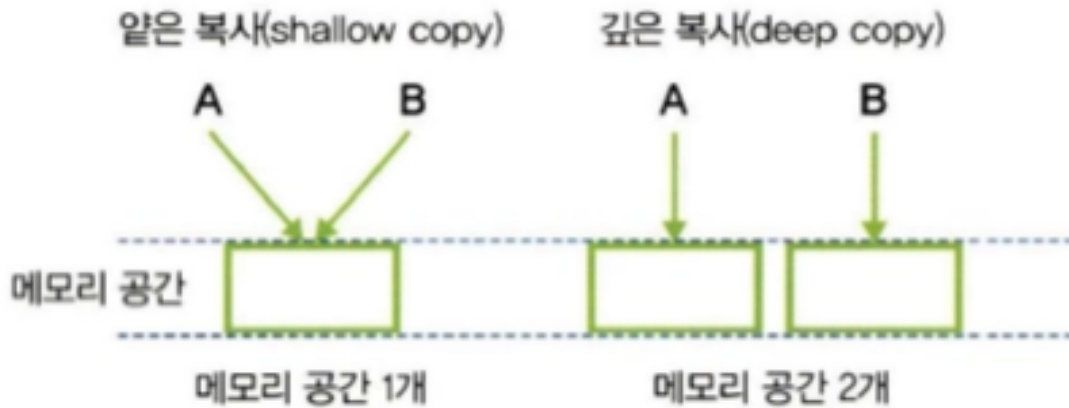
◦ 출력 결과

```
[[1, 2], 3]
[100, 3]
[[1, 2], 3]
[[1, 2, 4], 3]
[[1, 2], 3]
```

- copy_o에 [1, 2] 값을 100으로 변경했더니 copy_o만 바뀐 것은 동일

- [1, 2]에 4를 추가했더니 copy_o는 변경되었지만 original은 그대로

▼ 그림 6-14 얕은 복사와 깊은 복사



1. VGG 모델 정의

```
class VGG(nn.Module):
    def __init__(self, features, output_dim):
        super().__init__()
        self.features = features
        self.avgpool = nn.AdaptiveAvgPool2d(7)
        self.classifier = nn.Sequential(
            nn.Linear(512 * 7 * 7, 4096),
            nn.ReLU(inplace = True),
            nn.Dropout(0.5),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace = True),
            nn.Dropout(0.5),
            nn.Linear(4096, output_dim),
        )

    def forward(self, x):
        x = self.features(x)
        x = self.avgpool(x)
        h = x.view(x.shape[0], -1)
        x = self.classifier(h)
        return x, h
```

2. 모델 유형 정의

- VGG11, VGG13, VGG16, VGG19 모델 계층을 정리한 것
- 숫자(output, channel, 출력 채널) : Conv2d를 수행해라
- 출력 채널(output channel)이 다음 계층의 입력 채널(input channel)이 됨
- M: 최대 풀링을 수행해라

```
vgg11_config = [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M']
```

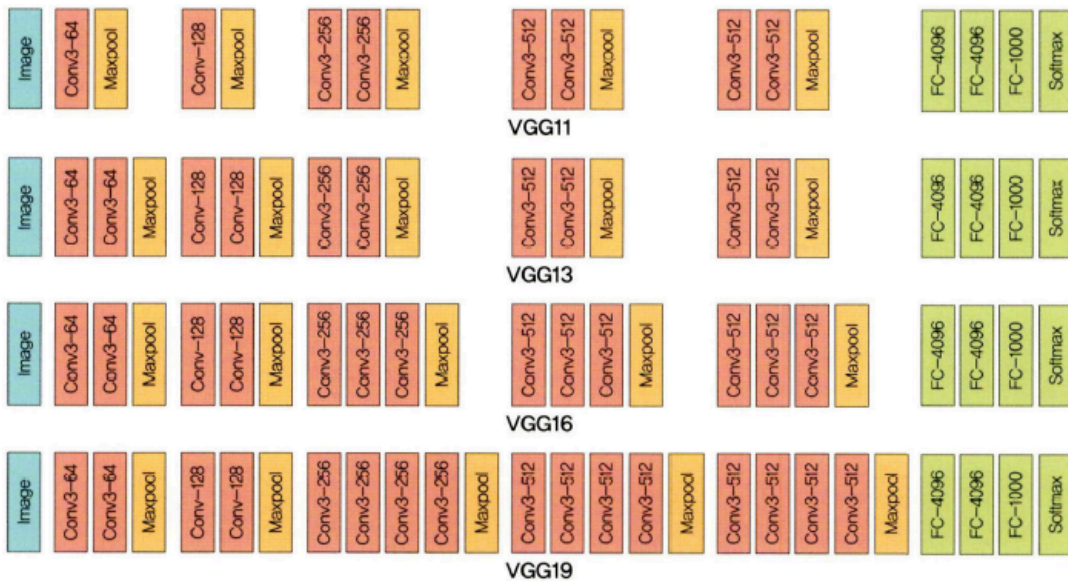
```
vgg13_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M']
```

```
vgg16_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M', 512, 512, 512, 'M']
```

```
vgg19_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512, 512, 'M', 512, 512, 512, 512, 'M']
```

- VGG11 = 8(합성곱층) + 3(풀링층) = 11(전체계층)
- VGG13 = 10(합성곱층) + 3(풀링층) = 13(전체계층)
- VGG16 = 13(합성곱층) + 3(풀링층) = 16(전체계층)
- VGG19 = 16(합성곱층) + 3(풀링층) = 19(전체계층)

▼ 그림 6-15 VGG의 다양한 모델에 대한 네트워크



3. VGG 계층 정의

- vgg11_config 사용함

```
def get_vgg_layers(config, batch_norm):
    layers = []
    in_channels = 3

    for c in config:
        assert c == 'M' or isinstance(c, int) #1
        if c == 'M':
            layers += [nn.MaxPool2d(kernel_size = 2)]
        else:
            conv2d = nn.Conv2d(in_channels, c, kernel_size
                                = 3, padding = 1)
            if batch_norm:
                layers += [conv2d, nn.BatchNorm2d(c), nn.Re
LU(inplace = True)]
            else:
                layers += [conv2d, nn.ReLU(inplace = True)]
            in_channels = c

    return nn.Sequential(*layers)
```

- 조건문 정의

- `assert c == 'M'` : `assert`: 가정 설정문 - 뒤의 조건이 True가 아니면 에러를 발생 시킴

```
a = 1 ; assert a == 11
```

- 조건이 True가 아니므로 오류 발생
- `isinstance(c, int)` : `isinstance`: 주어진 조건이 True인지 판단

```
print(isinstance(1, int)) #1이 integer인지 판단
print(isinstance(1, 2, int)) #1,2가 integer인지 판단
print(isinstance('deep learning', str)) #deep learning이
string인지 판단
```

- 출력 결과

True

False

True

- ∴ `assert c == 'M' or isinstance(c, int)`: `c`가 'M'이 아니거나 `int`가 아니라면 오류 발생

4. 모델 계층 생성

- `get_vgg_layers()` 함수 호출
- 여기서 배치 정규화(batch normalization)에 대한 계층도 추가

```
vgg11_layers = get_vgg_layers(vgg11_config, batch_norm = True)
```

- `batch_norm`(Batch Normalization): 데이터 평균을 0으로, 표준편차를 1로 분포시키는 것
 - 각 계층에서 입력 데이터 분포는 앞 계층에서 업데이트된 가중치에 따라 변함
→ 각 계층마다 변화되는 분포는 학습 속도를 늦출 뿐만 아니라 학습도 어렵게 함

5. VGG11 계층 확인

```
print(vgg11_layers)
```

- 출력 결과 확인하면 배치 정규화가 추가된 것을 확인할 수 있다

6. VGG11 전체에 대한 네트워크

```
OUTPUT_DIM = 2
model = VGG(vgg11_layers, OUTPUT_DIM).to(device)
print(model)
```

- 모델 전체에 대한 네트워크 확인 = vgg11_layer + VGG()에서 정의했던 완전연결층과 출력층((classifier): Sequential())이 포함된 구성

7. VGG11 사전 훈련된 모델 사용

- VGG11을 더 쉽게 사용하는 방법
 - 원래 VGG 모델은 사전 훈련된 모델 = 이미 누군가가 대용량의 이미지 데이터로 학습을 시키고, 최상을 상태로 튜닝을 거쳐 모든 사람이 사용할 수 있도록 공유한 사전 훈련된 모델
 - 다음과 같이 코드 한두 줄로 쉽게 가져올 수 있음

```
import torchvision.models as models
pretrained_model = models.vgg11_bn(pretrained = True) #1
print(pretrained_model)
```

- 배치 정규화가 적용된 사전 훈련된 VGG11 모델 사용 위해서 필요한 파라미터
 - vgg11_bn: VGG11 기본 모델에 배치 정규화가 적용된 모델을 사용
 - pretrained를 True로 설정하면 사전 훈련된 모델 사용

8. 이미지 데이터 전처리

```
train_transforms = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomRotation(5),
    transforms.RandomHorizontalFlip
```

```

(0.5),
                                transforms.ToTensor(),
                                transforms.Normalize(mean=[0.48
5, 0.456, 0.406],
                                std=[0.229, 0.224, 0.225])) #1

test_transforms = transforms.Compose([
                                transforms.Resize((256, 256)),
                                transforms.ToTensor(),
                                transforms.Normalize(mean=[0.48
5, 0.456, 0.406],
                                std=[0.229, 0.224, 0.225]))])

```

- 데이터 전처리에 사용되는 transforms, Compose에 대하여
 - transforms.Resize: 이미지를 주어진 크기로 재조정
 - transforms.RandomRotation: 5도 이하로 이미지 회전

9. ImageFolder 이용하여 데이터셋 불러오기

```

train_path = '/content/chap06/data/catanddog/train'
test_path = '/content/chap06/data/catanddog/test'

train_dataset = torchvision.datasets.ImageFolder(
    train_path,
    transform=train_transforms
) #1

test_dataset = torchvision.datasets.ImageFolder(
    test_path,
    transform=test_transforms
)

print(len(train_dataset)), print(len(test_dataset))

```

- ImageFolder에서 사용하는 파라미터 - 5장
- ImageFolder를 언제 사용하면 좋을까? 계층적인 폴더 구조 가지고 있는 데이터셋 불러올 때 사용

- 훈련과 테스트 데이터로 사용될 이미지 수
 - 출력 결과
 - 529
 - 12
 - (None, None)
 - 훈련 용도의 데이터 529개 사용 / 테스트 용도의 데이터 12개 사용

10. 훈련과 검증 데이터 분할

```
VALID_RATIO = 0.9
n_train_examples = int(len(train_dataset) * VALID_RATIO)
n_valid_examples = len(train_dataset) - n_train_examples

train_data, valid_data = data.random_split(train_dataset,
                                           [n_train_examples,
                                           n_valid_examples]) #1
```

- random_split(): 훈련과 검증 데이터셋 나누는 용도. 데이터가 데이터로더로 넘어간 이후에는 분리 불가능 → 데이터셋 단계에서 진행해야함
 - 첫 번째 파라미터(train_dataset): 분할에 사용될 데이터셋
 - 두 번째 파라미터([n_train_examples, n_valid_examples]): 훈련과 검증 데이터셋의 크기 지정. [훈련 데이터셋 크기, 검증 데이터셋 크기]

11. 검증 데이터 전처리

```
valid_data = copy.deepcopy(valid_data)
valid_data.dataset.transform = test_transforms
```

- 앞에서 훈련 데이터셋으로 분리된 valid_data를 valid_data 변수에 복사한 후 'test_transforms'로 전처리 적용

12. 훈련, 검증, 테스트 데이터셋 수 확인

```
print(f'Number of training examples: {len(train_data)}')
print(f'Number of validation examples: {len(valid_data)}')
print(f'Number of testing examples: {len(test_dataset)}')
```

- 출력 결과
 Number of training examples: 476
 Number of validation examples: 53
 Number of testing examples: 12
- 훈련용 476개, 검증용 53개, 테스트용 12개 데이터셋으로 구성된 것 확인 가능

13. 메모리로 데이터 불러오기

- 데이터로더 이용하여 데이터셋의 데이터를 메모리로 가져옴
- 데이터 가져올 때는 배치 크기만큼 나누어서 가져옴

```
BATCH_SIZE = 32
train_iterator = data.DataLoader(train_data,
                                 shuffle = True,
                                 batch_size = BATCH_SIZE)

valid_iterator = data.DataLoader(valid_data,
                                 batch_size = BATCH_SIZE)

test_iterator = data.DataLoader(test_dataset,
                                batch_size = BATCH_SIZE)
```

14. 옵티마이저와 손실 함수 정의

```
optimizer = optim.Adam(model.parameters(), lr = 1e-7)
criterion = nn.CrossEntropyLoss()

model = model.to(device)
criterion = criterion.to(device)
```

15. 모델 정확도 측정 함수

```
def calculate_accuracy(y_pred, y):
    top_pred = y_pred.argmax(1, keepdim = True)
    correct = top_pred.eq(y.view_as(top_pred)).sum() #1
    acc = correct.float() / y.shape[0]
    return acc
```

- 예측이 정답과 일치하는 경우 - 그 개수의 합을 correct 변수에 저장
 - eq: equal의 약자. 서로 같은지 비교하는 표현식
 - view_as(other): other의 텐서 크기를 사용하겠다는 의미
 → view_as(other) = view(other.size())
 ∴ y.view_as(top_pred): y에 대한 텐서 크기를 top_pred의 텐서 크기로 변경하겠다는 의미
 - sum(): 합계 구함 → 예측과 정답이 일치하는 것들의 개수를 합산하겠다는 의미

16. 모델 학습 함수 정의

```
def train(model, iterator, optimizer, criterion, device):
    epoch_loss = 0
    epoch_acc = 0

    model.train()
    for (x, y) in iterator:
        x = x.to(device)
        y = y.to(device)

        optimizer.zero_grad()
        y_pred, _ = model(x)
        loss = criterion(y_pred, y)
        acc = calculate_accuracy(y_pred, y)
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()
        epoch_acc += acc.item()
```

```
    return epoch_loss / len(iterator), epoch_acc / len(iterator)
```

17. 모델 성능 측정 함수

```
def evaluate(model, iterator, criterion, device):
    epoch_loss = 0
    epoch_acc = 0

    model.eval()
    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y = y.to(device)
            y_pred, _ = model(x)
            loss = criterion(y_pred, y)
            acc = calculate_accuracy(y_pred, y)
            epoch_loss += loss.item()
            epoch_acc += acc.item()
    return epoch_loss / len(iterator), epoch_acc / len(iterator)
```

18. 학습 시간 측정 함수

```
def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time
    elapsed_mins = int(elapsed_time / 60)
    elapsed_secs = int(elapsed_time - (elapsed_mins * 60))
    return elapsed_mins, elapsed_secs
```

19. 모델 학습

```
import time

EPOCHS = 5
```

```

best_valid_loss = float('inf')
for epoch in range(EPOCHS):
    start_time = time.monotonic()
    train_loss, train_acc = train(model, train_iterator, optimizer, criterion, device)
    valid_loss, valid_acc = evaluate(model, valid_iterator, criterion, device)

    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), 'VGG-model.pt')

    end_time = time.monotonic()
    epoch_mins, epoch_secs = epoch_time(start_time, end_time)

    print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_mins}m {epoch_secs}s')
    print(f'\tTrain Loss: {train_loss:.3f} | Train Acc: {train_acc*100:.2f}%')
    print(f'\tValid. Loss: {valid_loss:.3f} | Valid. Acc: {valid_acc*100:.2f}%')

```

- 훈련과 검증 데이터셋 이용한 모델 학습 결과

```

Epoch: 01 | Epoch Time: 0m 14s
    Train Loss: 0.704 | Train Acc: 46.73%
    Valid. Loss: 0.692 | Valid. Acc: 58.26%
Epoch: 02 | Epoch Time: 0m 8s
    Train Loss: 0.696 | Train Acc: 49.64%
    Valid. Loss: 0.694 | Valid. Acc: 44.12%
Epoch: 03 | Epoch Time: 0m 10s
    Train Loss: 0.695 | Train Acc: 50.62%
    Valid. Loss: 0.691 | Valid. Acc: 58.26%
Epoch: 04 | Epoch Time: 0m 9s
    Train Loss: 0.691 | Train Acc: 51.13%
    Valid. Loss: 0.689 | Valid. Acc: 53.50%
Epoch: 05 | Epoch Time: 0m 9s
    Train Loss: 0.701 | Train Acc: 49.73%
    Valid. Loss: 0.689 | Valid. Acc: 55.88%

```

- 성능이 좋진 않음. 데이터셋으로 사용되는 이미지 수와 에포크가 매우 적음
그러나 현재 데이터셋에서 에포크만 늘린다고 성능 좋아지진 않음 → 데이터도 함께 증가시켜 모델 학습 시켜야함

20. 테스트 데이터셋을 이용한 모델 성능 측정

- 앞에서 저장했던 모델(VGG-model.pt)를 불러와서 테스트 데이터셋에 대한 성능 측정

```
model.load_state_dict(torch.load('VGG-model.pt'))
test_loss, test_acc = evaluate(model, test_iterator, criterion, device)
print(f'Test Loss: {test_loss:.3f} | Test Acc: {test_acc*100:.2f}%')
```

- 출력 결과
Test Loss: 0.692 | Test Acc: 50.00%
- 결과가 좋지 않음..

21. 테스트 데이터셋을 이용한 모델의 예측 확인 함수

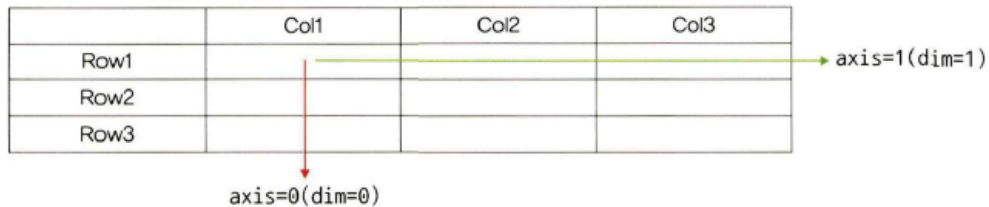
```
def get_predictions(model, iterator):
    model.eval()
    images = []
    labels = []
    probs = []

    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y_pred, _ = model(x)
            y_prob = F.softmax(y_pred, dim = -1)
            top_pred = y_prob.argmax(1, keepdim = True) #1
            images.append(x.cpu())
            labels.append(y.cpu())
            probs.append(y_prob.cpu())

    images = torch.cat(images, dim = 0) #2
```

```
labels = torch.cat(labels, dim = 0)
probs = torch.cat(probs, dim = 0)
return images, labels, probs
```

- argmax: 배열에서 가장 큰 값의 인덱스 찾을 때 사용
 - 첫 번째 파라미터(1): 행(axis=0) 또는 열(axis=1)을 따라 가장 큰 값의 색인 찾기
→ 1은 열을 따라 가장 큰 값의 색인을 찾겠다는 뜻
 - keepdim: keepdim=True의 경우: 출력 텐서를 입력과 동일한 크기로 유지하겠다는 의미
- torch.cat: 텐서 연결할 때 사용
- 차원(dim)



```
#torch.cat 예시
import torch
x = torch.Tensor([[1, 2, 3], [2, 3, 4]])
y = torch.Tensor([[4, 5, 6], [5, 6, 7]])

print(torch.cat([x], dim=0)) #행 기준(dim=0)으로 x를 이어붙임
print('-----')
print(torch.cat([x, y])) #x와 y를 결합하라. 출력결과=(4x3)형태
print('-----')
print(torch.cat([x, y], dim=0)) #행 기준(dim=0)으로 x와 y를
이어붙임/(4x3) 텐서가 됨
print('-----')
print(torch.cat([x, y], dim=1)) #열 기준(dim=1)으로 x와 y를
이어붙임/(2x6) 텐서가 됨
```

- 출력 결과


```
tensor([[1., 2., 3.],
        [2., 3., 4.]])
```

```

tensor([[1., 2., 3.],
        [2., 3., 4.],
        [4., 5., 6.],
        [5., 6., 7.]])

tensor([[1., 2., 3.],
        [2., 3., 4.],
        [4., 5., 6.],
        [5., 6., 7.]])

tensor([[1., 2., 3., 4., 5., 6.],
        [2., 3., 4., 5., 6., 7.]])

```

22. 예측 중에서 정확하게 예측한 것을 추출

- 앞에서 정의한 get_predictions() 함수의 반환값을 각각 images, labels, probs에 저장하여 모델이 정확하게 예측한 이미지를 추출함

```

images, labels, probs = get_predictions(model, test_iterator)
pred_labels = torch.argmax(probs, 1) #1
corrects = torch.eq(labels, pred_labels)
correct_examples = []

for image, label, prob, correct in zip(images, labels, probs, corrects):#2
    if correct:
        correct_examples.append((image, label, prob))

correct_examples.sort(reverse = True, key = lambda x: torch.max(x[2], dim = 0).values) #3

```

- argmax: max 최댓값 반환, argmax: 최댓값을 갖는 인덱스를 반환

```

x = torch.rand(5, 3)
print(x)
#임의의 텐서를 생성
print(torch.max(x)) #max 반환값

```

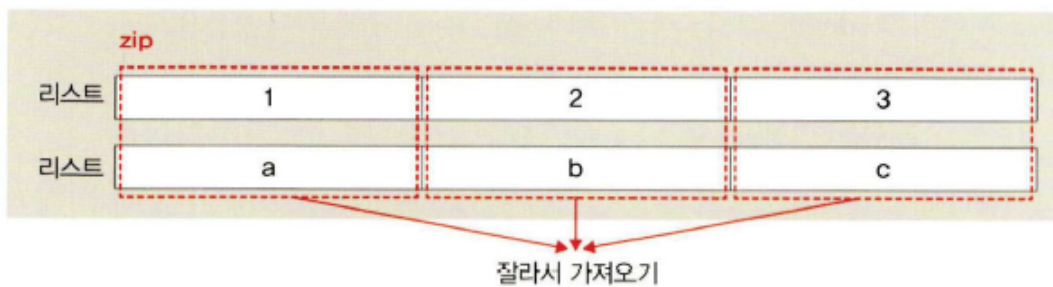
```
print('-----')
print(torch.argmax(x)) #argmax 반환값
```

- 출력 결과

tensor(0.9881)

tensor(14)

- zip(): 여러 개의 리스트(튜플) 합쳐서 새로운 튜플 타입으로 반환



```
a = [1, 2, 3]
b = ['a', 'b', 'c']
for x, y in zip(a, b):
    print(x, y)
```

- zip 적용 결과

1 a

2 b

3 c

- 데이터를 정렬하기 위해 sort() 메서드 사용

- reverse: 내림차순으로 정렬

```
x = [1, 7, 3, 9] #정렬되어 있지 않은 x라는 텐서 생성
x.sort(reverse=True)
print('x:', x)
```

- 출력 결과 - x가 내림차순으로 정렬됨

x: [9, 7, 3, 1]

- key: 데이터를 정렬할 때 key 값을 가지고 정렬하며 기본값은 오름차순. 람다: 일종의 함수. 일반적으로 함수는 def 함수명()처럼 사용하지만 람다함수는 함수명 없이도 사용 가능

♥ 그림 6-19 일반 함수와 람다 함수



- 람다 함수: 정의와 동시에 사용 가능, 함수명 X, 저장된 변수 X → 재사용 불가능

```
y = lamda x: x + 10
y(1)
```

- 출력 결과

11

```
#torch.max(x[2], dim=0) 의미
x = torch.rand([4, 4]) #(4x4) 크기의 임의의 텐서 생성
print(x)

max_elements, max_idx = torch.max(x, dim=0)
#torch.max 값을 가지고 오되 행을 기준으로 최댓값을 가져옴
print(max_elements)
print(max_idx)
```

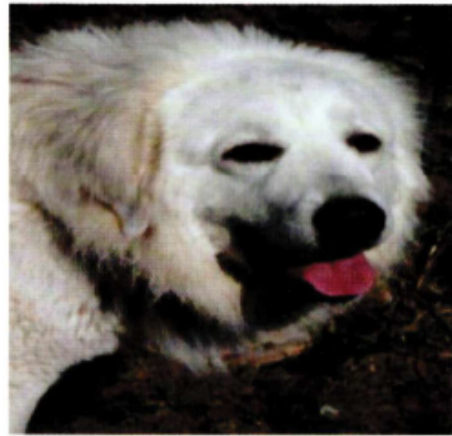
23. 이미지 출력을 위한 전처리

- 이미지 출력 위한 함수
- 앞에서 이미지 크기 및 전처리 과정 등을 거쳤기 때문에 이 상태에서 이미지 출력하면 왼쪽 그림임 → 오른쪽의 본래 이미지 출력하기 위해서는 다음과 같은 전처리 필요

♥ 그림 6-20 본래 이미지 색상이 왜곡됨



♥ 그림 6-21 본래 이미지



```
def normalize_image(image):  
    image_min = image.min()  
    image_max = image.max()  
    image.clamp_(min = image_min, max = image_max)  
    image.add_(-image_min).div_(image_max - image_min + 1e-  
5) #1  
    return image
```

- torch.add: 더하는 메서드
 - torch.add가 아닌 torch.add_를 사용

```
x = torch.tensor([1, 2])  
y = x.add(10) #torch.add 적용  
print(y)  
print(x is y)  
print('-----')  
y = x.add_(10) #torch.add_ 적용  
print(y)  
print(x is y)
```

- torch.add와 torch.add_의 결과는 같지만 / x가 y와 같은지(x is y) 물었을 때는 다른 값 출력함
- torch.add - 새로운 메모리 공간이 할당되어 저장됨 → x와 y가 같은지 물었을 때 False 결과 출력

- torch.add_ - 새로운 공간 할당 없이 기존 메모리에 위치한 값 대체
 - 메서드에 _ 표시: 기존의 메모리 공간에 있는 값을 새로운 값으로 대체하겠다

24. 모델이 정확하게 예측한 이미지 출력 함수

```
def plot_most_correct(correct, classes, n_images, normalize
= True):
    rows = int(np.sqrt(n_images))
    cols = int(np.sqrt(n_images))
    fig = plt.figure(figsize = (25, 20))
    for i in range(rows*cols):
        ax = fig.add_subplot(rows, cols, i+1)
        image, true_label, probs = correct[i]
        image = image.permute(1, 2, 0) #1
        true_prob = probs[true_label]
        correct_prob, correct_label = torch.max(probs, dim
= 0)

        true_class = classes[true_label]
        correct_class = classes[correct_label]

        if normalize:
            image = normalize_image(image)

        ax.imshow(image.cpu().numpy())
        ax.set_title(f'true label: {true_class} ({true_pro
b:.3f})\n' \
                    f'pred label: {correct_class} ({correc
t_prob:.3f})')
        ax.axis('off')

    fig.subplots_adjust(hspace = 0.4)
```

- image.permute: 축 변경시 사용

```
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print(x)
x.permute(1, 0) #차원(축) 0과 1 바꿈
```

- 출력 결과
- `tensor([[1, 2, 3],
[4, 5, 6]])`
`tensor([[1, 4],
[2, 5],
[3, 6]])`
 - 행과 열 변경됨

25. 예측 결과 이미지 출력

- 모델이 정확하게 예측한 이미지를 출력하기 위해 `plot_most_correct()` 함수 호출함
- 호출시 모델이 정확하게 예측한 이미지(`correct_examples`)에 대해 출력하도록 함

```
classes = test_dataset.classes
N_IMAGES = 5
plot_most_correct(correct_examples, classes, N_IMAGES)
```

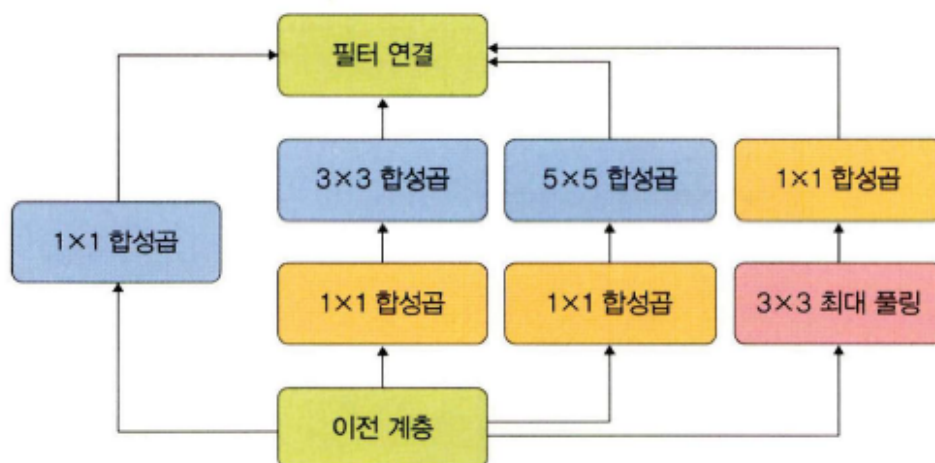


- 결과 정확도 높지 않음. 데이터셋 늘려준다면 정확도에 대한 성능 높아질 것

6.1.4 GoogLeNet

- 주어진 하드웨어 자원을 최대한 효율적으로 이용하면서 학습 능력은 극대화할 수 있는 깊고 넓은 신경망
- 인셉션(inception) 모듈 추가 → 깊고 넓은 신경망
 - 특징을 효율적으로 추출하기 위해 1×1 , 3×3 , 5×5 의 합성곱 연산을 각각 수행함
 - 3×3 최대 풀링 - 입력과 출력의 높이와 너비가 같아야 하므로 풀링 연산에서는 드물게 패딩을 추가해야함
- GoogLeNet에 적용된 해결방법 : 희소 연결(sparse connectivity)
 - CNN - 합성곱, 풀링, 완전연결층들이 서로 밀집(dense)(정교하고 뾰뾰하게)하게 연결되어있음
 - 희소 연결: 뾰뾰하게 연결된 신경망 대신 관련성(correlation)이 높은 노드끼리만 연결하는 방법
 - 연산량이 적어지며 과적합도 해결 가능

▼ 그림 6-23 GoogLeNet의 인셉션 모듈



- 인셉션 모듈의 네 가지 연산
 - 1×1 합성곱
 - 1×1 합성곱 + 3×3 합성곱
 - 1×1 합성곱 + 5×5 합성곱
 - 3×3 최대 풀링(maxpooling) + 1×1 합성곱(convolutional)

- 심층 신경망의 아키텍처에서 계층이 넓고(뉴런이 많고) 깊으면(계층이 많으면)
 - 인식률은 좋아짐
 - 과적합이나 기울기 소멸 문제(vanishing gradient problem)를 비롯한 학습 시간 지연과 연산 속도 등의 문제 발생
 - 특히, 합성곱 신경망에서 이러한 문제들 자주 발생함 → GoogLeNet(인셉션)으로 이러한 문제 해결 가능함